使用 CLAUDE CODE 构建

通过 MCP 将 Claude Code 连接到工具

复制页面

了解如何使用 Model Context Protocol 将 Claude Code 连接到您的工具。

Claude Code 可以通过 **Model Context Protocol (MCP)**（一个用于 AI 工具集成的开源标准）连接到数百个外部工具和数据源。MCP servers 为 Claude Code 提供对您的工具、数据库和 API 的访问权限。

使用 MCP 可以做什么

连接 MCP servers 后，您可以要求 Claude Code：

从问题跟踪器实现功能："添加 JIRA 问题 ENG-4521 中描述的功能，并在 GitHub 上创建 PR。"

分析监控数据："检查 Sentry 和 Statsig 以检查 ENG-4521 中描述的功能的使用情况。"

查询数据库："根据我们的 PostgreSQL 数据库，找到 10 个使用功能 ENG-4521 的随机用户的电子邮件。"

集成设计："根据在 Slack 中发布的新 Figma 设计更新我们的标准电子邮件模板"

自动化工作流："创建 Gmail 草稿，邀请这 10 个用户参加关于新功能的反馈会议。"

流行的 MCP servers

以下是一些您可以连接到 Claude Code 的常用 MCP servers：

⚠ 使用第三方 MCP servers 需自担风险 - Anthropic 尚未验证 所有这些 servers 的正确性或安全性。确保您信任正在安装的 MCP servers。 使用可能获取不受信任内容的 MCP servers 时要特别小心， 因为这些可能会使您面临提示注入风险。

Error loading MCP servers: Failed to fetch

Claude Code Docs

›

安装 MCP servers

MCP servers 可以根据您的需求以三种不同的方式配置：

选项1：添加远程 HTTP server

HTTP servers 是连接到远程 MCP servers 的推荐选项。这是云服务最广泛支持的传输方式。

```
# 基本语法
claude mcp add --transport http <name> <url>

# 真实示例：连接到 Notion
claude mcp add --transport http notion https://mcp.notion.com/mcp

# 带有 Bearer token 的示例
claude mcp add --transport http secure-api https://api.example.com/mcp \
  --header "Authorization: Bearer your-token"
```

选项2：添加远程 SSE server

⚠ SSE (Server-Sent Events) 传输已弃用。请在可用的地方使用 HTTP servers。

```
# 基本语法
claude mcp add --transport sse <name> <url>

# 真实示例：连接到 Asana
claude mcp add --transport sse asana https://mcp.asana.com/sse

# 带有身份验证标头的示例
claude mcp add --transport sse private-api https://api.company.com/sse \
  --header "X-API-Key: your-key-here"
```

Claude Code Docs

›

```
# 基本语法
claude mcp add [options] <name> -- <command> [args...]

# 真实示例：添加 Airtable server
claude mcp add --transport stdio --env AIRTABLE_API_KEY=YOUR_KEY airtable \
  -- npx -y airtable-mcp-server
```

重要：选项顺序

所有选项（ `--transport` 、 `--env` 、 `--scope` 、 `--header` ）必须在 server 名称**之前**。然后 `--` （双破折号）将 server 名称与传递给 MCP server 的命令和参数分开。

例如：

`claude mcp add --transport stdio myserver -- npx server` → 运行 `npx server`

`claude mcp add --transport stdio --env KEY=value myserver -- python server.py --port 8080` → 在环境中使用 `KEY=value` 运行 `python server.py --port 8080`

这可以防止 Claude 的标志与 server 的标志之间的冲突。

管理您的 servers

配置后，您可以使用这些命令管理您的 MCP servers：

```
# 列出所有配置的 servers
claude mcp list

# 获取特定 server 的详细信息
claude mcp get github

# 删除 server
claude mcp remove github

# （在 Claude Code 中）检查 server 状态
/mcp
```

Claude Code Docs

›

提示：

使用 `--scope` 标志指定配置的存储位置：

- `local` （默认）：仅在当前项目中对您可用（在较旧版本中称为 `project` ）
- `project` ：通过 `.mcp.json` 文件与项目中的每个人共享
- `user` ：在所有项目中对您可用（在较旧版本中称为 `global` ）

使用 `--env` 标志设置环境变量（例如， `--env KEY=value` ）

使用 MCP_TIMEOUT 环境变量配置 MCP server 启动超时（例如， `MCP_TIMEOUT=10000 claude` 设置 10 秒超时）

当 MCP 工具输出超过 10,000 个 tokens 时，Claude Code 将显示警告。要增加此限制，请设置 `MAX_MCP_OUTPUT_TOKENS` 环境变量（例如， `MAX_MCP_OUTPUT_TOKENS=50000` ）

使用 `/mcp` 对需要 OAuth 2.0 身份验证的远程 servers 进行身份验证

Windows 用户：在本机 Windows（不是 WSL）上，使用 `npx` 的本地 MCP servers 需要 `cmd /c` 包装器以确保正确执行。

```
# 这创建了 Windows 可以执行的 command="cmd"
claude mcp add --transport stdio my-server -- cmd /c npx -y @some/packa
```

没有 `cmd /c` 包装器，您会遇到"连接已关闭"错误，因为 Windows 无法直接执行 `npx` 。（有关 `--` 参数的说明，请参阅上面的注释。）

Plugin 提供的 MCP servers

Plugins 可以捆绑 MCP servers，在启用 plugin 时自动提供工具和集成。Plugin MCP servers 的工作方式与用户配置的 servers 相同。

Plugin MCP servers 的工作原理：

- Plugins 在 plugin 根目录的 `.mcp.json` 中或在 `plugin.json` 中内联定义 MCP servers
- 启用 plugin 时，其 MCP servers 会自动启动

Claude Code Docs

›

在 plugin 根目录的 `.mcp.json` 中：

```
{
  "database-tools": {
    "command": "${CLAUDE_PLUGIN_ROOT}/servers/db-server",
    "args": ["--config", "${CLAUDE_PLUGIN_ROOT}/config.json"],
    "env": {
      "DB_URL": "${DB_URL}"
    }
  }
}
```

或在 `plugin.json` 中内联：

```
{
  "name": "my-plugin",
  "mcpServers": {
    "plugin-api": {
      "command": "${CLAUDE_PLUGIN_ROOT}/servers/api-server",
      "args": ["--port", "8080"]
    }
  }
}
```

Plugin MCP 功能：

- **自动生命周期**：Servers 在 plugin 启用时启动，但您必须重新启动 Claude Code 以应用 MCP server 更改 (启用或禁用)
- **环境变量**：使用 `${CLAUDE_PLUGIN_ROOT}` 表示 plugin 相对路径
- **用户环境访问**：访问与手动配置的 servers 相同的环境变量
- **多种传输类型**：支持 stdio、SSE 和 HTTP 传输 (传输支持可能因 server 而异)

Plugin servers 在列表中出现，并带有指示它们来自 plugins 的指示符。

Plugin MCP servers 的优势：

- **捆绑分发**：工具和 servers 打包在一起
- **自动设置**：无需手动 MCP 配置
- **团队一致性**：安装 plugin 时每个人都获得相同的工具

有关使用 plugins 捆绑 MCP servers 的详细信息，请参阅 **plugin 组件参考**。

MCP 安装范围

MCP servers 可以在三个不同的范围级别配置，每个级别用于管理 server 可访问性和共享的不同目的。了解这些范围可以帮助您确定为特定需求配置 servers 的最佳方式。

本地范围

本地范围的 servers 代表默认配置级别，存储在项目路径下的 `~/.claude.json` 中。这些 servers 仅对您私有，仅在当前项目目录中工作时可访问。此范围非常适合个人开发 servers、实验配置或包含不应共享的敏感凭据的 servers。

> MCP servers 的"本地范围"术语与一般本地设置不同。MCP 本地范围的 servers 存储在 `~/.claude.json`（您的主目录）中，而一般本地设置使用 `.claude/settings.local.json`（在项目目录中）。有关设置文件位置的详细信息，请参阅 **Settings**。

```
# 添加本地范围的 server（默认）
claude mcp add --transport http stripe https://mcp.stripe.com

# 显式指定本地范围
claude mcp add --transport http stripe --scope local https://mcp.stripe.com
```

Claude Code Docs

›

```
# 添加项目范围的 server
claude mcp add --transport http paypal --scope project https://mcp.paypal.com/m
```

生成的 `.mcp.json` 文件遵循标准化格式：

```
{
  "mcpServers": {
    "shared-server": {
      "command": "/path/to/server",
      "args": [],
      "env": {}
    }
  }
}
```

出于安全原因，Claude Code 在使用来自 `.mcp.json` 文件的项目范围的 servers 之前会提示批准。如果您需要重置这些批准选择，请使用 `claude mcp reset-project-choices` 命令。

用户范围

用户范围的 servers 存储在 `~/.claude.json` 中，提供跨项目可访问性，使其在您机器上的所有项目中可用，同时对您的用户帐户保持私有。此范围适用于个人实用程序 servers、开发工具或您在不同项目中经常使用的服务。

```
# 添加用户 server
claude mcp add --transport http hubspot --scope user https://mcp.hubspot.com/ar
```

选择正确的范围

根据以下条件选择您的范围：

Claude Code Docs

>

MCP servers 存储在哪里?

用户和本地范围: `~/.claude.json` (在 `mcpServers` 字段或项目路径下)
项目范围: 项目根目录中的 `.mcp.json` (检入源代码控制)
托管: 系统目录中的 `managed-mcp.json` (请参阅 **托管 MCP 配置**)

范围层次结构和优先级

MCP server 配置遵循清晰的优先级层次结构。当具有相同名称的 servers 存在于多个范围时，系统通过优先考虑本地范围的 servers 来解决冲突，然后是项目范围的 servers，最后是用户范围的 servers。此设计确保个人配置可以在需要时覆盖共享配置。

`.mcp.json` 中的环境变量扩展

Claude Code 支持 `.mcp.json` 文件中的环境变量扩展，允许团队共享配置，同时为特定于机器的路径和 API 密钥等敏感值保持灵活性。

支持的语法:

`${VAR}` - 扩展为环境变量 `VAR` 的值

`${VAR:-default}` - 如果设置了 `VAR` ，则扩展为 `VAR` ，否则使用 `default`

扩展位置: 环境变量可以在以下位置扩展:

`command` - server 可执行文件路径

`args` - 命令行参数

`env` - 传递给 server 的环境变量

`url` - 对于 HTTP server 类型

`headers` - 对于 HTTP server 身份验证

带有变量扩展的示例:

Claude Code Docs

```
      "url": "${API_BASE_URL:-https://api.example.com}/mcp",
      "headers": {
        "Authorization": "Bearer ${API_KEY}"
      }
    }
  }
}
```

如果未设置所需的环境变量且没有默认值，Claude Code 将无法解析配置。

实际示例

示例：使用 Sentry 监控错误

```
# 1. 添加 Sentry MCP server
claude mcp add --transport http sentry https://mcp.sentry.dev/mcp

# 2. 使用 /mcp 对您的 Sentry 帐户进行身份验证
> /mcp

# 3. 调试生产问题
> "What are the most common errors in the last 24 hours?"
> "Show me the stack trace for error ID abc123"
> "Which deployment introduced these new errors?"
```

示例：连接到 GitHub 进行代码审查

```
> /mcp
# 为 GitHub 选择"身份验证"

# 3. 现在您可以要求 Claude 使用 GitHub
> "Review PR #456 and suggest improvements"
> "Create a new issue for the bug we just found"
> "Show me all open PRs assigned to me"
```

示例：查询您的PostgreSQL数据库

```
# 1. 使用您的连接字符串添加数据库 server
claude mcp add --transport stdio db -- npx -y @bytebase/dbhub \
  --dsn "postgresql://readonly:pass@prod.db.com:5432/analytics"

# 2. 自然地查询您的数据库
> "What's our total revenue this month?"
> "Show me the schema for the orders table"
> "Find customers who haven't made a purchase in 90 days"
```

使用远程MCP servers进行身份验证

许多基于云的 MCP servers 需要身份验证。Claude Code 支持 OAuth 2.0 以实现安全连接。

1. **添加需要身份验证的 server**

   例如：

   ```
   claude mcp add --transport http sentry https://mcp.sentry.dev/mcp
   ```

2. **在 Claude Code 中使用 /mcp 命令**

然后按照浏览器中的步骤登录。

提示：

- 身份验证 tokens 被安全存储并自动刷新
- 使用 `/mcp` 菜单中的"清除身份验证"撤销访问权限
- 如果浏览器没有自动打开，请复制提供的 URL 并手动打开
- 如果浏览器重定向在身份验证后失败并出现连接错误，请将浏览器地址栏中的完整回调 URL 粘贴到 Claude Code 中出现的 URL 提示中
- OAuth 身份验证适用于 HTTP servers

使用预配置的 OAuth 凭据

某些 MCP servers 不支持自动 OAuth 设置。如果您看到"不兼容的身份验证 server：不支持动态客户端注册"之类的错误，server 需要预配置的凭据。首先通过 server 的开发者门户注册 OAuth 应用程序，然后在添加 server 时提供凭据。

1. **使用 server 注册 OAuth 应用程序**

   通过 server 的开发者门户创建应用程序，并记下您的客户端 ID 和客户端密钥。

   许多 servers 还需要重定向 URI。如果是这样，请选择一个端口并以 `http://localhost:PORT/callback` 的格式注册重定向 URI。在下一步中使用 `--callback-port` 时使用相同的端口。

2. **使用您的凭据添加 server**

   选择以下方法之一。用于 `--callback-port` 的端口可以是任何可用的端口。它只需要与您在上一步中注册的重定向 URI 匹配。

   claude mcp add | **claude mcp add-json** | **CI / env var**

```
--client-id your-client-id --client-secret --callback-port 8080 \
my-server https://mcp.example.com/mcp
```

3 **在 Claude Code 中进行身份验证**

在 Claude Code 中运行 `/mcp` 并按照浏览器登录流程进行操作。

提示：

- 客户端密钥被安全存储在您的系统钥匙链 (macOS) 或凭据文件中，而不是在您的配置中
- 如果 server 使用没有密钥的公共 OAuth 客户端，仅使用 `--client-id` 而不使用 `--client-secret`
- 这些标志仅适用于 HTTP 和 SSE 传输。它们对 stdio servers 没有影响
- 使用 `claude mcp get <name>` 验证为 server 配置了 OAuth 凭据

覆盖 OAuth 元数据发现

如果您的 MCP server 在标准 OAuth 元数据端点（`/.well-known/oauth-authorization-server`）上返回错误，但公开了工作的 OIDC 端点，您可以告诉 Claude Code 直接从您指定的 URL 获取 OAuth 元数据，绕过标准发现链。

在 `.mcp.json` 中您的 server 配置的 `oauth` 对象中设置 `authServerMetadataUrl`：

Claude Code Docs

```
      "url": "https://mcp.example.com/mcp",
      "oauth": {
        "authServerMetadataUrl": "https://auth.example.com/.well-known/openid-c
      }
    }
  }
}
```

URL 必须使用 `https://` 。此选项需要 Claude Code v2.1.64 或更高版本。

从 JSON 配置添加 MCP servers

如果您有 MCP server 的 JSON 配置，您可以直接添加它：

1 **从 JSON 添加 MCP server**

```
# 基本语法
claude mcp add-json <name> '<json>'

# 示例：使用 JSON 配置添加 HTTP server
claude mcp add-json weather-api '{"type":"http","url":"https://api.weathe

# 示例：使用 JSON 配置添加 stdio server
claude mcp add-json local-weather '{"type":"stdio","command":"/path/to/we

# 示例：添加带有预配置 OAuth 凭据的 HTTP server
claude mcp add-json my-server '{"type":"http","url":"https://mcp.example.
```

Claude Code Docs

›

提示：

确保 JSON 在您的 shell 中正确转义

JSON 必须符合 MCP server 配置架构

您可以使用 `--scope user` 将 server 添加到您的用户配置而不是项目特定的配置

从 Claude Desktop 导入 MCP servers

如果您已在 Claude Desktop 中配置了 MCP servers，您可以导入它们：

1. **从 Claude Desktop 导入 servers**

```
# 基本语法
claude mcp add-from-claude-desktop
```

2. **选择要导入的 servers**

运行命令后，您将看到一个交互式对话框，允许您选择要导入的 servers。

3. **验证 servers 已导入**

```
claude mcp list
```

提示：

此功能仅在 macOS 和 Windows Subsystem for Linux (WSL) 上有效

使用来自 Claude.ai 的 MCP servers

如果您已使用 **Claude.ai** 帐户登录 Claude Code，您在 Claude.ai 中添加的 MCP servers 会自动在 Claude Code 中可用：

1. **在 Claude.ai 中配置 MCP servers**

   在 **claude.ai/settings/connectors** 添加 servers。在 Team 和 Enterprise 计划中，仅管理员可以添加 servers。

2. **对 MCP server 进行身份验证**

   在 Claude.ai 中完成任何必需的身份验证步骤。

3. **在 Claude Code 中查看和管理 servers**

   在 Claude Code 中，使用命令：

   ```
   # 在 Claude Code 中，查看所有 MCP servers，包括 Claude.ai 的
   > /mcp
   ```

   Claude.ai servers 在列表中出现，并带有指示它们来自 Claude.ai 的指示符。

要在 Claude Code 中禁用 claude.ai MCP servers，请将 `ENABLE_CLAUDEAI_MCP_SERVERS` 环境变量设置为 `false`：

```
ENABLE_CLAUDEAI_MCP_SERVERS=false claude
```

Claude Code Docs

›

```
# 启动 claude 作为 stdio MCP server
claude mcp serve
```

您可以通过将此配置添加到 claude_desktop_config.json 在 Claude Desktop 中使用它：

```
{
  "mcpServers": {
    "claude-code": {
      "type": "stdio",
      "command": "claude",
      "args": ["mcp", "serve"],
      "env": {}
    }
  }
}
```

配置可执行文件路径： `command` 字段必须引用 Claude Code 可执行文件。如果 `claude` 命令不在您的系统 PATH 中，您需要指定可执行文件的完整路径。

要查找完整路径：

```
which claude
```

然后在您的配置中使用完整路径：

```
      "command": "/full/path/to/claude",
      "args": ["mcp", "serve"],
      "env": {}
    }
  }
}
```

没有正确的可执行文件路径，您会遇到 `spawn claude ENOENT` 之类的错误。

提示：

server 提供对 Claude 的工具 (如 View、Edit、LS 等) 的访问权限。

在 Claude Desktop 中，尝试要求 Claude 读取目录中的文件、进行编辑等。

请注意，此 MCP server 仅向您的 MCP 客户端公开 Claude Code 的工具，因此您自己的客户端负责为单个工具调用实现用户确认。

MCP 输出限制和警告

当 MCP 工具产生大量输出时，Claude Code 可以帮助管理 token 使用情况，以防止压倒您的对话上下文：

- **输出警告阈值**：当任何 MCP 工具输出超过 10,000 个 tokens 时，Claude Code 显示警告
- **可配置限制**：您可以使用 `MAX_MCP_OUTPUT_TOKENS` 环境变量调整最大允许的 MCP 输出 tokens
- **默认限制**：默认最大值为 25,000 个 tokens

要为产生大量输出的工具增加限制：

```
# 为 MCP 工具输出设置更高的限制
export MAX_MCP_OUTPUT_TOKENS=50000
claude
```

Claude Code Docs

›

处理大量日志文件或调试信息

> ⚠ 如果您经常遇到特定 MCP servers 的输出警告，请考虑增加限制或配置 server 以对其响应进行分页或过滤。

使用MCP资源

MCP servers 可以公开您可以使用 @ 提及引用的资源，类似于您引用文件的方式。

引用MCP资源

1. **列出可用资源**

   在您的提示中键入 `@` 以查看来自所有连接的 MCP servers 的可用资源。资源与文件一起出现在自动完成菜单中。

2. **引用特定资源**

   使用格式 `@server:protocol://resource/path` 引用资源：

   ```
   > Can you analyze @github:issue://123 and suggest a fix?
   ```

   ```
   > Please review the API documentation at @docs:file://api/authentication
   ```

3. **多个资源引用**

   您可以在单个提示中引用多个资源：

Claude Code Docs

›

提示：

引用时会自动获取资源并将其作为附件包含

资源路径在 @ 提及自动完成中可进行模糊搜索

当 servers 支持时，Claude Code 会自动提供列出和读取 MCP 资源的工具

资源可以包含 MCP server 提供的任何类型的内容 (文本、JSON、结构化数据等)

使用 MCP Tool Search 进行扩展

当您配置了许多 MCP servers 时，工具定义可能会消耗上下文窗口的很大一部分。MCP Tool Search 通过动态按需加载工具而不是预加载所有工具来解决此问题。

工作原理

当您的 MCP 工具描述会消耗超过 10% 的上下文窗口时，Claude Code 会自动启用 Tool Search。您可以 **调整此阈值** 或完全禁用 tool search。触发时：

1. MCP 工具被延迟而不是预先加载到上下文中
2. Claude 使用搜索工具在需要时发现相关的 MCP 工具
3. 仅将 Claude 实际需要的工具加载到上下文中
4. MCP 工具从您的角度继续完全按照之前的方式工作

对于 MCP server 作者

如果您正在构建 MCP server，启用 Tool Search 时 server 说明字段会变得更有用。Server 说明可以帮助 Claude 理解何时搜索您的工具，类似于 **skills** 的工作方式。

添加清晰、描述性的 server 说明，说明：

您的工具处理的任务类别

Claude 何时应搜索您的工具

Claude Code Docs

›

Tool search 默认以自动模式运行，这意味着仅当您的 MCP 工具定义超过上下文阈值时才激活。如果您的工具很少，它们会正常加载而不使用 tool search。此功能需要支持 `tool_reference` 块的模型：Sonnet 4 及更高版本，或 Opus 4 及更高版本。Haiku 模型不支持 tool search。

使用 `ENABLE_TOOL_SEARCH` 环境变量控制 tool search 行为：

值	行为
`auto`	当 MCP 工具超过 10% 的上下文时激活 (默认)
`auto:<N>`	在自定义阈值激活，其中 `<N>` 是百分比 (例如， `auto:5` 表示 5%)
`true`	始终启用
`false`	禁用，所有 MCP 工具预先加载

```
# 使用自定义 5% 阈值
ENABLE_TOOL_SEARCH=auto:5 claude

# 完全禁用 tool search
ENABLE_TOOL_SEARCH=false claude
```

或在您的 **settings.json `env` 字段** 中设置值。

您还可以使用 `disallowedTools` 设置专门禁用 MCPSearch 工具：

```
{
  "permissions": {
    "deny": ["MCPSearch"]
  }
}
```

使用MCP提示作为命令

MCP servers 可以公开在 Claude Code 中作为命令可用的提示。

Claude Code Docs

›

键入 `/` 以查看所有可用的命令，包括来自 MCP servers 的命令。MCP 提示以 `/mcp__servername__promptname` 的格式出现。

2 **执行不带参数的提示**

```
> /mcp__github__list_prs
```

3 **执行带有参数的提示**

许多提示接受参数。在命令后面以空格分隔的方式传递它们：

```
> /mcp__github__pr_review 456
```

```
> /mcp__jira__create_issue "Bug in login flow" high
```

提示：

MCP 提示从连接的 servers 动态发现
参数根据提示的定义参数进行解析
提示结果直接注入到对话中
Server 和提示名称被规范化 (空格变为下划线)

托管 MCP 配置

对于需要对 MCP servers 进行集中控制的组织，Claude Code 支持两个配置选项：

Claude Code Docs

›

这些选项允许 IT 管理员：

控制 MCP servers 员工可以访问：在整个组织中部署一组标准化的已批准 MCP servers

防止未授权的 MCP servers：限制用户添加未批准的 MCP servers

完全禁用 MCP：如果需要，完全删除 MCP 功能

选项1：使用 managed-mcp.json 的独占控制

部署 `managed-mcp.json` 文件时，它对所有 MCP servers 进行**独占控制**。用户无法添加、修改或使用除此文件中定义的 MCP servers 之外的任何 MCP servers。这是希望完全控制的组织的最简单方法。

系统管理员将配置文件部署到系统范围的目录：

macOS： `/Library/Application Support/ClaudeCode/managed-mcp.json`

Linux 和 WSL： `/etc/claude-code/managed-mcp.json`

Windows： `C:\Program Files\ClaudeCode\managed-mcp.json`

> 这些是系统范围的路径（不是用户主目录，如 `~/Library/...`），需要管理员权限。它们设计为由 IT 管理员部署。

`managed-mcp.json` 文件使用与标准 `.mcp.json` 文件相同的格式：

Claude Code Docs

```
      "url": "https://api.githubcopilot.com/mcp/"
    },
    "sentry": {
      "type": "http",
      "url": "https://mcp.sentry.dev/mcp"
    },
    "company-internal": {
      "type": "stdio",
      "command": "/usr/local/bin/company-mcp-server",
      "args": ["--config", "/etc/company/mcp-config.json"],
      "env": {
        "COMPANY_API_URL": "https://internal.company.com"
      }
    }
  }
}
```

选项2：使用允许列表和拒绝列表的基于策略的控制

管理员可以允许用户配置自己的 MCP servers，同时对允许的 servers 进行限制，而不是进行独占控制。此方法在 **托管设置文件** 中使用 `allowedMcpServers` 和 `deniedMcpServers` 。

> **在选项之间选择**：当您想要部署固定的 servers 集合而不进行用户自定义时，使用选项 1（ `managed-mcp.json` ）。当您想要允许用户在策略约束内添加自己的 servers 时，使用选项 2（允许列表/拒绝列表）。

限制选项

允许列表或拒绝列表中的每个条目可以通过三种方式限制 servers：

1. **按 server 名称**（ `serverName` ）：匹配 server 的配置名称
2. **按命令**（ `serverCommand` ）：匹配用于启动 stdio servers 的确切命令和参数
3. **按 URL 模式**（ `serverUrl` ）：匹配带有通配符支持的远程 server URL

重要：每个条目必须恰好具有 `serverName` 、 `serverCommand` 或 `serverUrl` 之一。

Claude Code Docs

```
    // 按 server 名称允许
    { "serverName": "github" },
    { "serverName": "sentry" },

    // 按确切命令允许（对于 stdio servers）
    { "serverCommand": ["npx", "-y", "@modelcontextprotocol/server-filesystem"]
    { "serverCommand": ["python", "/usr/local/bin/approved-server.py"] },

    // 按 URL 模式允许（对于远程 servers）
    { "serverUrl": "https://mcp.company.com/*" },
    { "serverUrl": "https://*.internal.corp/*" }
  ],
  "deniedMcpServers": [
    // 按 server 名称阻止
    { "serverName": "dangerous-server" },

    // 按确切命令阻止（对于 stdio servers）
    { "serverCommand": ["npx", "-y", "unapproved-package"] },

    // 按 URL 模式阻止（对于远程 servers）
    { "serverUrl": "https://*.untrusted.com/*" }
  ]
}
```

基于命令的限制如何工作

精确匹配：

> 命令数组必须**精确**匹配 - 命令和所有参数的顺序正确
>
> 示例：`["npx", "-y", "server"]` 将**不**匹配 `["npx", "server"]` 或 `["npx", "-y", "server", "--flag"]`

Stdio server 行为：

> 当允许列表包含**任何** `serverCommand` 条目时，stdio servers **必须**匹配其中之一
>
> Stdio servers 在存在命令限制时无法仅按名称通过

Claude Code Docs

›

URL 的匹配

如果不存在 URL 条目，远程 servers 回退到基于名称的匹配

命令限制不适用于远程 servers

基于 URL 的限制如何工作

URL 模式使用 `*` 支持通配符以匹配任何字符序列。这对于允许整个域或子域很有用。

通配符示例：

`https://mcp.company.com/*` - 允许特定域上的所有路径

`https://*.example.com/*` - 允许 example.com 的任何子域

`http://localhost:*/*` - 允许 localhost 上的任何端口

远程 server 行为：

当允许列表包含**任何** `serverUrl` 条目时，远程 servers **必须**匹配其中一个 URL 模式

远程 servers 在存在 URL 限制时无法仅按名称通过

这确保管理员可以强制执行允许的远程端点

示例：仅 URL 允许列表

示例：仅命令允许列表

示例：混合名称和命令允许列表

示例：仅名称允许列表

允许列表行为（`allowedMcpServers`）

拒绝列表行为（`deniedMcpServers`）

`undefined`（默认）：没有 servers 被阻止

空数组 `[]`：没有 servers 被阻止

条目列表：指定的 servers 在所有范围内被显式阻止

重要说明

选项 1 和选项 2 可以组合：如果 `managed-mcp.json` 存在，它具有独占控制，用户无法添加 servers。允许列表/拒绝列表仍然适用于托管 servers 本身。

拒绝列表具有绝对优先级：如果 server 匹配拒绝列表条目（按名称、命令或 URL），即使它在允许列表上也会被阻止

基于名称、基于命令和基于 URL 的限制一起工作：如果 server 匹配**任何**名称条目、命令条目或 URL 模式，它就会通过（除非被拒绝列表阻止）

> **使用 `managed-mcp.json` 时**：用户无法通过 `claude mcp add` 或配置文件添加 MCP servers。`allowedMcpServers` 和 `deniedMcpServers` 设置仍然适用于过滤实际加载的托管 servers。

此页面对您有帮助吗？ 是 否

‹ 编程使用 故障排除 ›

Claude Code Docs

Trust center

Transparency

Learn

Courses

MCP connectors

Customer stories

Engineering blog

Events

Powered by Claude

Service partners

Startups program

Terms and policies

Privacy policy

Disclosure policy

Usage policy

Commercial terms

Consumer terms